MLOPS PROJECT REPORT

GUARDIANOPS: A SELF-HEALING MONITORING AND GOVERNANCE LAYER FOR A FRAUD-DETECTION SERVICE

Submitted in partial fulfilment
of the
requirement for the award of the degree of

BACHELOR OF TECHNOLOGY (B. Tech)

in

Computer Science and Engineering (AIML)

Submitted by

Kasmya Bhatia
(23FE10CAI00010)

Submitted to



MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING,
SCHOOL OF COMPUTER SCIENCE & ENGINEERING,
MANIPAL UNIVERSITY JAIPUR,
RAJASTHAN, INDIA-303007

OCTOBER, 2026

ACKNOWLEDGMENTS

First and foremost, I wish to express my sincere thanks and gratitude to my esteemed guide [Dr Jayant Mahawar] for the thoughtful reviews and valuable guidance that contributed to the successful completion of this project. I am also grateful to the open-source communities behind FastAPI, SciPy, MLflow, Prometheus, Grafana, Docker and pytest, whose documentation made it possible to build and verify the system with standard, inspectable tools. Next I would like to tender my sincere thanks to Prof. [Dr Deepak Panwar] (Head of Computer Science and Engineering Department), the Department project coordinator and the faculty members whose assistance was sought during the work, for their co-operation and encouragement.

[Kasmya Bhatia]
(University Registration No. [23FE10CAI00010])

ABSTRACT

A machine-learning service in production can fail in two very different ways. A crash is loud: the service stops answering and any health check notices. A silent failure is quiet: the service keeps returning well-formed answers with a normal status code and a normal latency, but the answers have become wrong, so ordinary uptime monitoring reports nothing while the business is harmed. This report documents GuardianOps, a small MLOps platform built to detect both kinds of failure in a fraud-detection scoring service, to respond to each under an explicit governance policy, and to leave an auditable record of every incident.

The system is made of three FastAPI services and a browser dashboard. The monitored fraud-api scores transactions with a deliberately simple rule-based scorer. A chaos-injector provokes a crash, a latency fault or a silent corruption on demand. The guardian service polls the scorer every two seconds, freezes a 200-sample baseline of scores, and every five seconds compares a rolling window of 60 scores with that baseline using the two-sample Kolmogorov-Smirnov test and the Population Stability Index (PSI). A crash is recovered automatically under policy P-001 and timed; a silent failure opens an incident that waits for a human to approve the rollback under policy P-002. Each incident is stored in SQLite, written up as a markdown post-mortem and logged as an MLflow run.

Verification is reported as measured. Thirty-eight automated tests pass (16 for fraud-api and 22 for guardian), a 35-check endpoint audit passes, and a scripted end-to-end scenario passes: no incident opens during 40 seconds of healthy traffic, injected corruption opens a pending incident, the system does not roll back by itself, and an approval resolves the incident in about 6.5 seconds. Testing also exposed a serious defect in the first detector, which raised a false incident in every simulated healthy hour; replacing it with a rule that requires statistical significance and three consecutive checks reduced the simulated false-alarm rate to 1.0% of hours. The rule-based scorer itself is weak (recall 25% on a labelled replay of 500 transactions) and the Docker deployment could not be run in the verification environment; both facts are reported in Chapter 4.

The principal tools were Python 3.11, FastAPI, SciPy, NumPy, SQLite, MLflow, Prometheus, Grafana, Docker Compose, GitHub Actions, pytest, Playwright, HTML, CSS and JavaScript.

Contents

			Page No
Acknowledgment			i
Abstract			ii
List Of Figures			iv
List Of Tables			v
Chapter 1		**INTRODUCTION**	**1**
	1.1	Scope of the Work	1
	1.2	Product Scenarios	3
	1.3	Organization of Report	3
Chapter 2		**REQUIREMENT ANALYSIS**	**4**
	2.1	Functional Requirements	4
	2.2	Non-functional Requirements	4
	2.3	Use Case Scenarios	5
	2.4	Other Software Engineering Methodologies	6
Chapter 3		**SYSTEM DESIGN**	**8**
	3.1	Design Goals	8
	3.2	System Architecture	8
	3.3	Detailed Design Methodologies	10
Chapter 4		**WORK DONE**	**15**
	4.1	Development Environment	15
	4.2	Implementation Modules	15
	4.3	Results and Discussion	22
	4.4	Individual Contribution of Project Members	23
Chapter 5		**CONCLUSION AND FUTURE WORK**	**25**
	5.1	Conclusion	25
	5.2	Future Scope of Work	25
	5.3	Proposed Workplan of the Project	25
REFERENCES			**27**
ANNEXURES			**28**

LIST OF FIGURES

Figure No	Figure Title	Page No
1.1	Overview of the work streams carried out during the project	2
2.1	Use-case diagram of GuardianOps	5
3.1	Architecture of GuardianOps as implemented in the repository	8
3.2	Route structure of the services (30 routes in three FastAPI apps)	9
3.3	Composition of the dashboard from its blocks	10
3.4	Request and response flow of a silent-failure incident	11
3.5	Control flow of the drift check	13
4.1	Dashboard in the healthy state (local run, commit 54b90c8)	16
4.2	A crash: no-signal region on the trace, incident and decision log (local run)	17
4.3	Silent-failure incident awaiting approval (banner, ledger and trace; local run)	18
4.4	Approval card with the first two sections expanded (local run)	19
4.5	Auto-generated incident post-mortem after approval (local run)	20
4.6	Development and debugging workflow used during the project	20
4.7	Build and deployment flow defined by the workflow and the compose file	22
4.8	Replay of 500 labelled transactions: confusion matrix and metrics (local run)	23

LIST OF TABLES

Table No	Table Title	Page No
1.1	Project particulars (source: repository history and course allotment)	1
2.1	Functional requirements of GuardianOps	4
2.2	Non-functional requirements	4
2.3	Use-case summaries	5
2.4	Traceability of functional requirements	6
3.1	Modules of the guardian service and their responsibilities	9
3.2	Key API endpoints and their observed behaviour	11
3.3	Fields of the incidents table	12
4.1	Technology stack and responsibilities	15
4.2	Defect log: symptom, cause and action	20
4.3	Verification performed and observed results	22
4.4	Limitations identified and suggested improvements	23
4.5	Mapping of my contribution to modules and evidence	24
5.1	Skills and learning outcomes	25
5.2	Proposed future work	25
5.3	Proposed workplan for the next phase	26
A.1	Abbreviations used in this report	29

1. INTRODUCTION

Machine-learning models are now embedded in services that make decisions about money, health and safety, and the discipline of keeping such services working after they are deployed has acquired its own name, MLOps. A well-known engineering paper observed that the learning code is only a small part of a real machine-learning system and that most of the effort and most of the hidden cost lies in the surrounding infrastructure for configuration, monitoring and serving [10]. A deployed model does not fail only by stopping. It can also keep running while the world, the input data or the model itself changes underneath it, and its answers slowly or suddenly stop being right.

GuardianOps is a small, complete platform that makes this difference concrete. It wraps a fraud-detection scoring service with a monitor that distinguishes a crash from a silent failure, applies a written governance policy to each, and records every incident as evidence. The scorer is a service that returns a fraud score between 0 and 1 for a card transaction and labels it ok, review or fraud. The word fraud-detection is used for its realism: a bank that starts declining every card because the model silently broke has a real and expensive incident, even though every HTTP response was successful.

Two kinds of failure are treated. A crash is loud and easy to detect, and the safe response, restarting the process, needs no judgement. A silent failure is detected only by looking at the distribution of the service's outputs, and the response, rolling the model back, is itself a risky action on a live system. GuardianOps therefore automates the first response and deliberately withholds the second until a named human approves it. This split between what a machine may do alone and what needs an accountable person is the governance idea of the project.

The work was carried out as an individual project for the MLOps course between 3 September and 25 October 2026. Table 1.1 summarises the particulars. The source code, the tests, the continuous integration workflow and this report are in the public repository github.com/pradhanashwarya2122/GuardianOps [1]; the version described in this report is commit 54b90c8.

Table 1.1: Project particulars (source: repository history and course allotment)

Data and honesty note. All transactions used by the system are synthetic. The scorer is a hand-written rule set and not a trained model, as stated in the repository diary, because the project is about operating a model and not about training one. Accuracy figures in this report describe that rule set on synthetic labelled data and must not be read as the performance of a production fraud model.

1.1 Scope of the Work

The scope was the operational layer around a scoring service: the monitor, the governance logic, the failure-injection tools, the dashboard, the observability stack and the automated checks that make the claims of the system verifiable.

1.1.1 Project Statement

The project statement is to build and verify a monitoring and governance layer that can tell a crash from a silent failure in a fraud-scoring service, respond to each under an explicit policy, and give an operator a clear and honest picture of what happened. Honest is used in a practical sense: the dashboard shows live values and not decoration, the thresholds can be changed at run time and take effect, and the automated tests are able to fail.

1.1.2 Objectives

- To implement a scoring service whose behaviour can be switched between normal, slow and silently corrupted by a separate chaos-injector service.

- To detect silent output drift with a statistical rule that has a measured false-alarm rate, and to detect a crash through missed health checks.
- To enforce two written policies: automatic restart for a crash (P-001) and human approval before rollback for a silent failure (P-002).
- To record each incident in a database, a markdown post-mortem and an MLflow run so that it can be audited.
- To present the live state in a dashboard that reads real endpoints, with controls that act on the system.
- To verify the whole system with unit tests, an endpoint audit and an end-to-end scenario that run in continuous integration.

1.1.3 In scope and out of scope

In scope were the three FastAPI services, the dashboard, the SQLite store, the Prometheus and Grafana configuration, the Docker Compose file, the Windows launcher, the test suites and the GitHub Actions workflow. Out of scope were training a real fraud model, any use of real cardholder data, authentication and role-based access control for the approval action, multi-instance deployment of the monitored service and any claim about production-grade accuracy. The approval selector on the dashboard is a label for the audit trail and not an authentication mechanism; Section 4.3 lists this as a limitation.

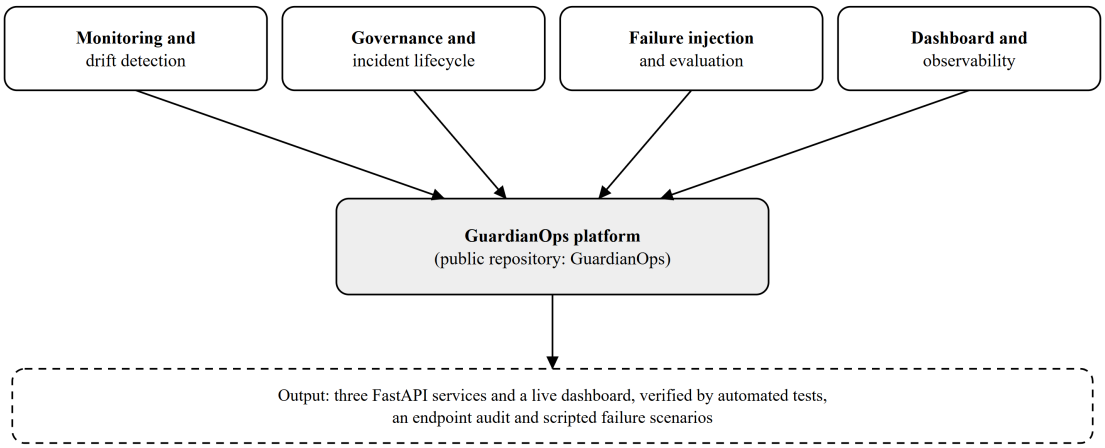


Figure 1.1: Overview of the work streams carried out during the project

Figure 1.1 gives an overview of the four work streams. They were not strictly sequential: the first dashboard was written alongside the services, and the testing work in October rewrote part of the detector after the tests showed that it raised false alarms.

1.2 Product Scenarios

A product scenario describes a realistic situation in which the system is used. Four scenarios drove the design.

1.2.1 Scenario 1: the scoring service crashes

An on-call engineer is watching the dashboard when the scoring process dies. Within about four seconds the guardian service sees two consecutive failed health checks, classifies the event as a crash, opens an incident and records it. Under policy P-001 no human decision is needed, because restarting a process is safe, so the incident is closed automatically once the service answers again and the time to recovery is stored.

1.2.2 Scenario 2: the scorer silently starts returning wrong scores

A bad deployment or a broken feature pipeline shifts the scores upward. Every response is HTTP 200 and fast. Only the distribution of the scores has moved. Guardian detects the shift, opens a pending incident and raises a banner, but it does not roll back, because under policy P-002 a named person must authorise the rollback. The approval card explains the evidence in plain language and in numbers.

1.2.3 Scenario 3: an auditor reviews what happened

A model owner or auditor later asks which incidents occurred, how long each took to resolve and who approved the action. The incident table, the markdown post-mortem of each incident and the MLflow run created for it answer these questions without anyone having to reconstruct events from logs.

1.2.4 Scenario 4: an evaluator replays labelled transactions

A reviewer who doubts that the scorer handles realistic transactions presses the replay control. The service streams a bundled file of 500 labelled transactions through the same scoring code and returns a confusion matrix and the precision, recall, F1 score and accuracy, so the quality of the scorer is measured and not assumed.

These scenarios also explain a deliberate property of the design. The scorer is simple on purpose, and the dataset is synthetic on purpose, so that every failure that is demonstrated is one that the operator provoked and the tests can reproduce.

1.3 Organization of Report

Chapter 1 introduces the problem, the project and its scope. Chapter 2 analyses the requirements, expresses them as functional and non-functional requirements and use cases, and states the software engineering methods used. Chapter 3 presents the system design: design goals, architecture and the detailed design of the major mechanisms, including the drift rule. Chapter 4 describes the work done, including the development environment, each implementation module, the results of verification and the individual contribution. Chapter 5 concludes the report, lists the future scope and gives a proposed workplan. The references and annexures follow.

2. REQUIREMENT ANALYSIS

The requirements were derived from the course handout, from the behaviour expected of an operational monitor and from defects found while testing the running system. This chapter records them in the form that a software engineering course expects, so that the design in Chapter 3 and the verification in Chapter 4 can be traced back to them. Where a requirement is inferred from the code, it is described as observed behaviour.

2.1 Functional Requirements

Table 2.1 lists the functional requirements. Each has an identifier that is reused in the verification table of Chapter 4.

Table 2.1: Functional requirements of GuardianOps

ID	Requirement	Source of the requirement
FR1	The fraud-api shall score a transaction (amount, merchant category, hour, country risk) and return a score in [0, 1], a label of ok, review or fraud and a model version, rejecting invalid input.	fraud-api/app/main.py
FR2	The chaos-injector shall switch fraud-api between normal, slow and corrupt modes and shall be able to terminate the process and reset it.	chaos-injector/app/main.py
FR3	The guardian shall capture a 200-sample baseline, then compare a rolling 60-sample window with it every five seconds using the KS test and PSI, and open a silent-failure incident only after three consecutive significant checks.	monitor.py, drift.py
FR4	The guardian shall classify two consecutive failed health checks as a crash and record the time to recovery (policy P-001).	monitor.py, recovery.py
FR5	A silent-failure incident shall stay in the pending_approval state until POST /approve/{id} is called; the rollback shall not occur earlier (policy P-002).	recovery.py, main.py
FR6	Every incident shall be stored in SQLite, written as a markdown post-mortem and logged as an MLflow run.	store.py, reports.py, mlflow_logger.py
FR7	Thresholds and intervals shall be readable and changeable at run time through GET and POST /config, with values clamped to safe ranges.	main.py (config)
FR8	The fraud-api shall replay a bundled labelled CSV through the scoring code and return a confusion matrix and quality metrics.	main.py (replay)
FR9	The dashboard shall show live status, the score trace, the session ledger, the approval card and the post-mortem, and shall provide controls for the chaos actions, the forced drift check, the replay and the approval.	frontend/index.html

Requirement FR3 deserves a note because it changed during the project. The first version opened an incident when PSI reached 0.20 on two checks. Testing showed that this raised a false alarm on healthy traffic, and Section 3.3.6 and Module 2 in Chapter 4 describe why the requirement now demands statistical significance as well.

2.2 Non-functional Requirements

Non-functional requirements describe how well the system must perform its functions. Table 2.2 lists those that guided the work. They are stated as design intentions, and the verification chapter states which of them were actually checked.

Table 2.2: Non-functional requirements

ID	Quality	Requirement
NFR1	Accuracy of alerts	A healthy system shall rarely raise an incident, and the false-alarm rate shall be measured and reported.
NFR2	Safety	A rollback of a live model shall require explicit approval; an approval that cannot be carried out shall leave the incident pending and shall be reported as an error.
NFR3	Observability	Services shall expose Prometheus metrics, and the key gauges shall appear in a provisioned Grafana dashboard.
NFR4	Reliability	Endpoints shall return meaningful HTTP status codes (200, 400, 404, 422, 502) and JSON bodies, including on failure.
NFR5	Security	Report file names shall be validated so that the report endpoint cannot read files outside the reports directory.
NFR6	Portability	The system shall start with one command on Windows (a batch file) or with docker compose up on any host.
NFR7	Testability	Every service shall have automated tests, and an end-to-end scenario shall run without manual steps.
NFR8	Traceability	Every change shall be committed with a descriptive message.

2.3 Use Case Scenarios

A use case captures an interaction between an actor and the system that delivers value to the actor. Figure 2.1 shows the use cases of GuardianOps. There are two actors. The operator or approver is the primary actor and uses the dashboard. The chaos injector is a secondary actor, a test harness that provokes failures and does not read any result.

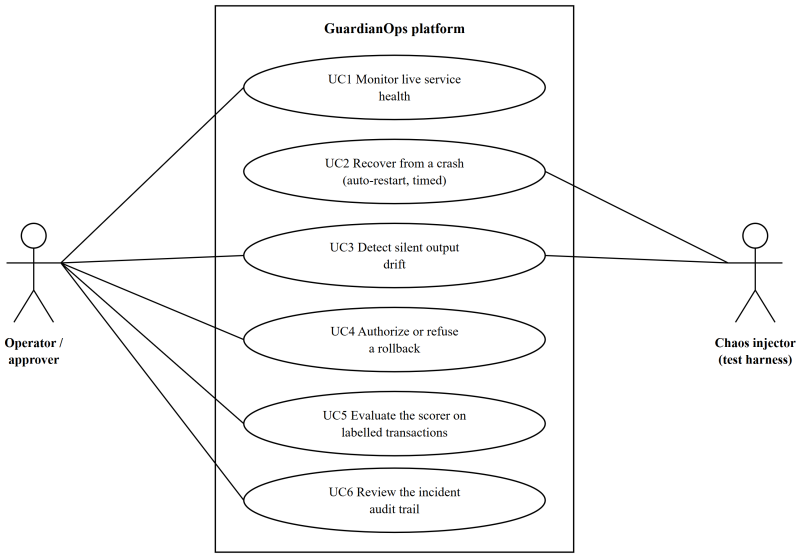


Figure 2.1: Use-case diagram of GuardianOps

Table 2.3 summarises each use case with its main flow and the alternative flows that the implementation handles.

Table 2.3: Use-case summaries

Use case	Main flow	Alternative flows handled
UC1 Monitor live service health	Guardian polls /health and /score; the dashboard polls Guardian every 1.5 seconds and draws the trace, uptime, latency and confidence.	When fraud-api is unreachable the dashboard shows DOWN and the trace marks a no-signal region.
UC2 Recover from a crash	Two missed health checks open a crash incident; Docker restarts the container (or the operator restarts the process); Guardian records the time to recovery.	No human approval is required (P-001).
UC3 Detect silent output drift	Window is full; KS test and PSI against the baseline; three consecutive significant checks open a pending incident.	Fewer than 60 samples: no check. A minor or stable verdict resets the counter.
UC4 Authorize or refuse a rollback	Operator reads the approval card and calls approve; fraud-api is reset to normal; the incident is resolved.	Unknown or already resolved incident: HTTP 400. fraud-api unreachable: HTTP 502 and the incident stays pending.
UC5 Evaluate the scorer on labelled transactions	Replay streams 500 rows through the scorer and returns a confusion matrix and metrics.	Missing dataset file: HTTP 404.
UC6 Review the incident audit trail	Operator opens the latest post-mortem or lists earlier reports; each resolved incident is also an MLflow run.	No incident yet: HTTP 404 on the latest report. Invalid report name: HTTP 404.

Table 2.4 traces each functional requirement to the use cases it supports and to the module in Chapter 4 in which it was addressed.

Table 2.4: Traceability of functional requirements

Requirement	Use case	Implemented and discussed in
FR1	UC1	Module 1 (fraud-api)
FR2	UC2, UC3	Module 5 (chaos-injector)
FR3	UC3	Module 2 (monitor and drift detection), Section 3.3.6
FR4	UC2	Module 2 (monitor and drift detection)
FR5	UC4	Module 3 (governance and incident lifecycle)
FR6	UC6	Module 3 (governance and incident lifecycle)
FR7	UC3	Module 2 and Module 6 (dashboard settings)
FR8	UC5	Module 1 (replay) and Module 6 (replay modal)
FR9	UC1 to UC6	Module 6 (dashboard), Module 7 (observability)

2.4 Other Software Engineering Methodologies

The project followed lightweight, practical methodologies rather than a heavyweight process. They are described here because the evidence for them is visible in the repository history, and because they explain several design choices.

2.4.1 Iterative and incremental development

Work was divided into small increments, each producing a runnable system: the three services and a first dashboard, then the scope visualisation, then the replay dataset and runtime configuration, then the approval card, then the redesigned interface, then the hardening and testing described in this report. After each increment the system was started and exercised, which limited the damage of any one faulty change.

2.4.2 Service-oriented design

The monitored system, the failure injector and the monitor are separate processes with their own ports and HTTP contracts. A monitor that shares a process with the thing it monitors would die with it, so the separation is a requirement of the problem and not only a matter of taste. Each service is small enough to read in one sitting: the three services together are about 1,000 lines of Python.

2.4.3 Version control with Git

All work was tracked with Git and published to GitHub. A commit is a snapshot with an author, a timestamp and a message, and the history can be queried to establish who changed what and when. This report uses that history as its evidence of contribution, and the commit identifiers quoted in Chapter 4 can be checked by anyone with access to the public repository [1], [13].

2.4.4 Test-driven defect triage

Defect triage is the practice of reproducing a problem, classifying it, writing a test that fails because of it, applying the smallest fix and re-running everything. The loop is shown in Chapter 4. The most

important defect of the project, the false-alarm detector, was found only because a test written to assert a plausible property of the healthy-traffic statistic failed.

2.4.5 Continuous integration and static checks

A GitHub Actions workflow runs both unit-test suites, boots the three services as processes and runs the endpoint audit and the end-to-end scenario, and builds the Docker images [8]. Automation makes the verification repeatable by a reviewer who has not seen the project before.

3. SYSTEM DESIGN

This chapter describes how GuardianOps is organised. The description is verified against the source code at commit 54b90c8, and the diagrams show only structures that exist in that code. The Docker Compose description is verified by configuration validation only, because the container engine was not available in the verification environment (Section 4.3).

3.1 Design Goals

The design follows from the requirements of Chapter 2. Five goals were pursued.

- **Separation of monitor and monitored.** The monitor, the scorer and the fault injector are different processes, so a crash of the scorer cannot take the monitor down.
- **Governed automation.** A safe action is automated and a risky action waits for an accountable person; the policy is written down in the code, the dashboard and each post-mortem.
- **Measured detection.** The detector's false-alarm rate and detection delay are measured by a reproducible simulation and reported, instead of being asserted.
- **Evidence by default.** Every incident leaves a database row, a post-mortem and an MLflow run.
- **Reproducibility.** Dependencies are pinned in requirements files, the dataset is generated from a fixed seed, and one batch file or one compose command starts the system.

3.2 System Architecture

Figure 3.1 shows the architecture as implemented. The operator's browser loads a static dashboard and then talks directly to the three services over HTTP. The fraud-api is the monitored service. The chaos-injector changes the mode of fraud-api through its admin routes, which store the mode in a small shared state file that fraud-api reads on every request. The guardian polls fraud-api, evaluates the drift rule, applies the policies, and writes incidents to SQLite and post-mortems to disk. Prometheus scrapes the metrics endpoints of fraud-api and guardian, Grafana reads Prometheus, and MLflow receives one run per resolved incident [4], [5], [6].

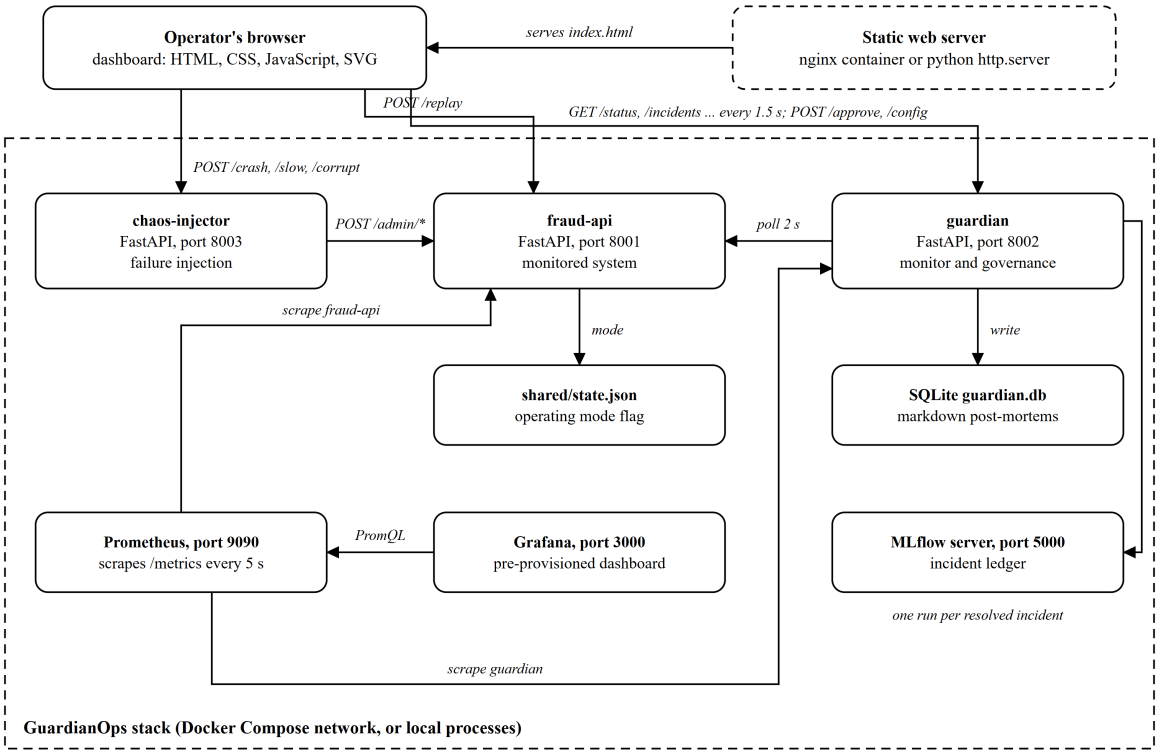


Figure 3.1: Architecture of GuardianOps as implemented in the repository

The outer frame in Figure 3.1 marks the stack that Docker Compose creates, which can equally be run as local processes by the batch launcher. The browser and the static web server are outside it. Arrows show the direction of the call: the browser calls the services, the guardian calls fraud-api, and the observability tools call the metrics endpoints.

The three services expose 30 routes in total, shown in Figure 3.2. The fraud-api has nine routes, the guardian sixteen and the chaos-injector five. Each service is a FastAPI application, which generates an interactive schema page automatically [2].

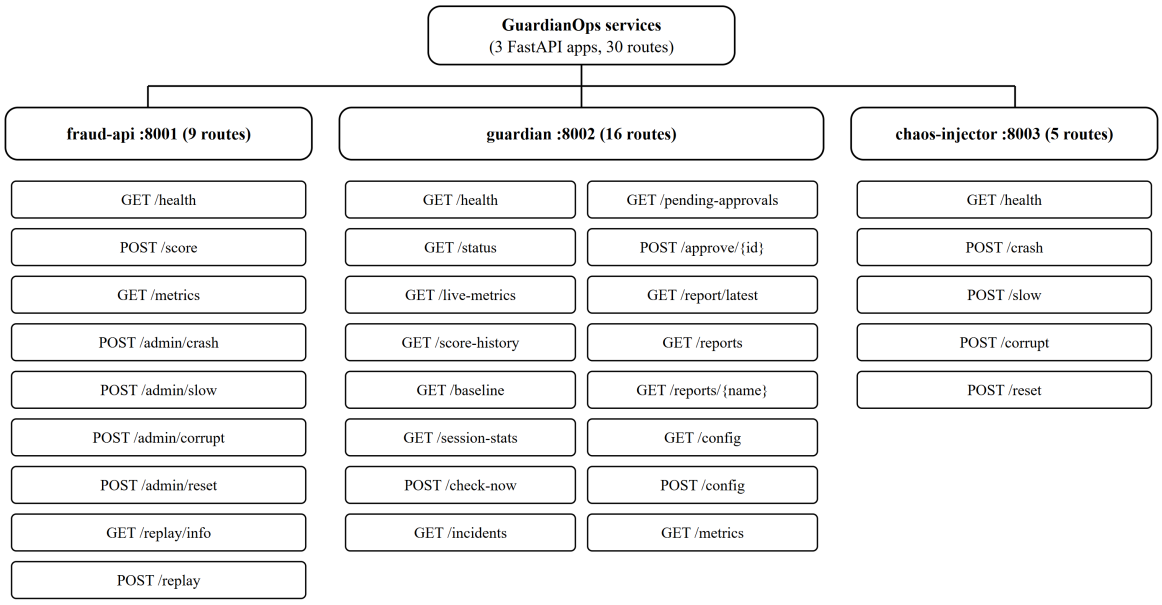


Figure 3.2: Route structure of the services (30 routes in three FastAPI apps)

Table 3.1 lists the modules of the guardian service and their responsibilities. The descriptions are taken from reading each file.

Table 3.1: Modules of the guardian service and their responsibilities

Module	Responsibility
config.py	All tunables in one place: poll interval 2 s, drift check interval 5 s, window 60, baseline 200, PSI warn 0.10 and alert 0.20, three consecutive alerts, two health misses, and the policy identifier and text.
monitor.py	The monitor loop: health polling, baseline capture, rolling window, drift check scheduling, counters for consecutive misses and alerts, live metrics and score history for the dashboard, and the Prometheus gauges.
drift.py	PSI over ten baseline-quantile bins, the two-sample KS test, the verdict rule and the confidence gauge.
recovery.py	The two policy handlers: crash (P-001) and silent failure (P-002), the reasoning text, and the approval action with its failure handling.
store.py	SQLite schema and queries for the incidents table; session statistics.
reports.py	Renders the markdown post-mortem of an incident.
mlflow_logger.py	Logs a resolved incident as an MLflow run with tags, metrics and the report as an artifact; failure is non-fatal.
main.py	FastAPI routes, runtime configuration with clamping, report-name validation and the Prometheus endpoint.

3.3 Detailed Design Methodologies

3.3.1 Dashboard composition

The dashboard is a single HTML file with its styles and script inline, so that it can be served by any static web server and opened without a build step. As Figure 3.3 shows, it presents twelve blocks in a fixed order. Each block reads one or two endpoints, and none of the numbers it shows is hard-coded.

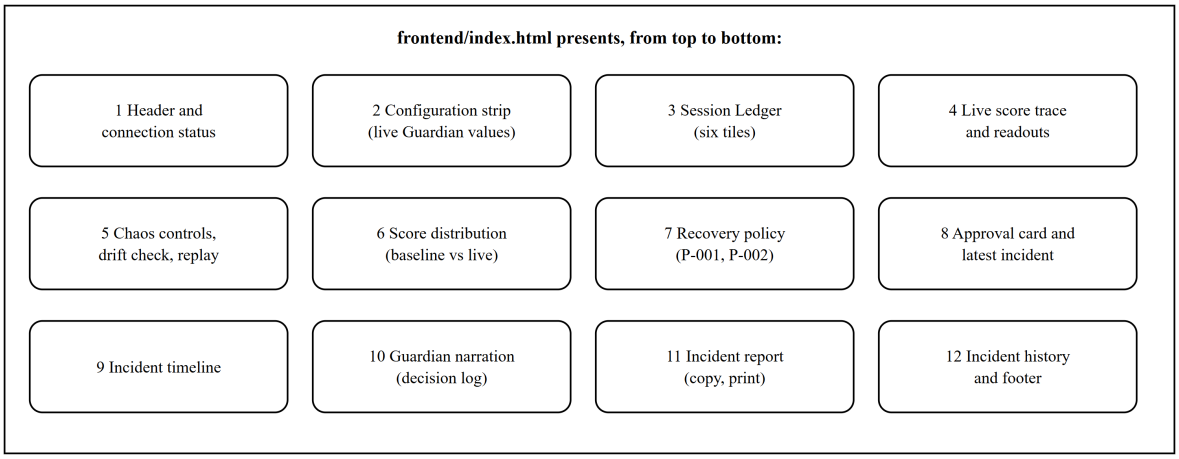


Figure 3.3: Composition of the dashboard from its blocks

3.3.2 Service boundaries and shared state

The chaos-injector and the fraud-api need to agree on the current mode, but they are separate processes. The agreement is a small JSON file, shared/state.json, written by the admin routes of fraud-api and read by the scoring route. The file also holds the artificial latency, and a startup hook resets it to normal so that a mode left over from a previous run cannot poison a new baseline. This is a simple design with a known cost: the services must share a volume, and a multi-instance deployment would need a real shared store.

3.3.3 Request and response design

All routes exchange JSON over HTTP. Figure 3.4 follows one silent-failure scenario from injection to approval. The operator's click reaches the chaos-injector, which switches fraud-api to corrupt mode. The guardian keeps polling, sees the shifted scores, and after three consecutive significant checks writes a pending incident. The dashboard finds it on its next poll and raises a banner. Only when the operator calls the approve route does the guardian reset fraud-api and resolve the incident. The status codes follow the semantics of HTTP: 200 for success, 400 for a request that cannot be applied to the current state, 404 for a missing resource, 422 for invalid input and 502 for a failed upstream call [12].

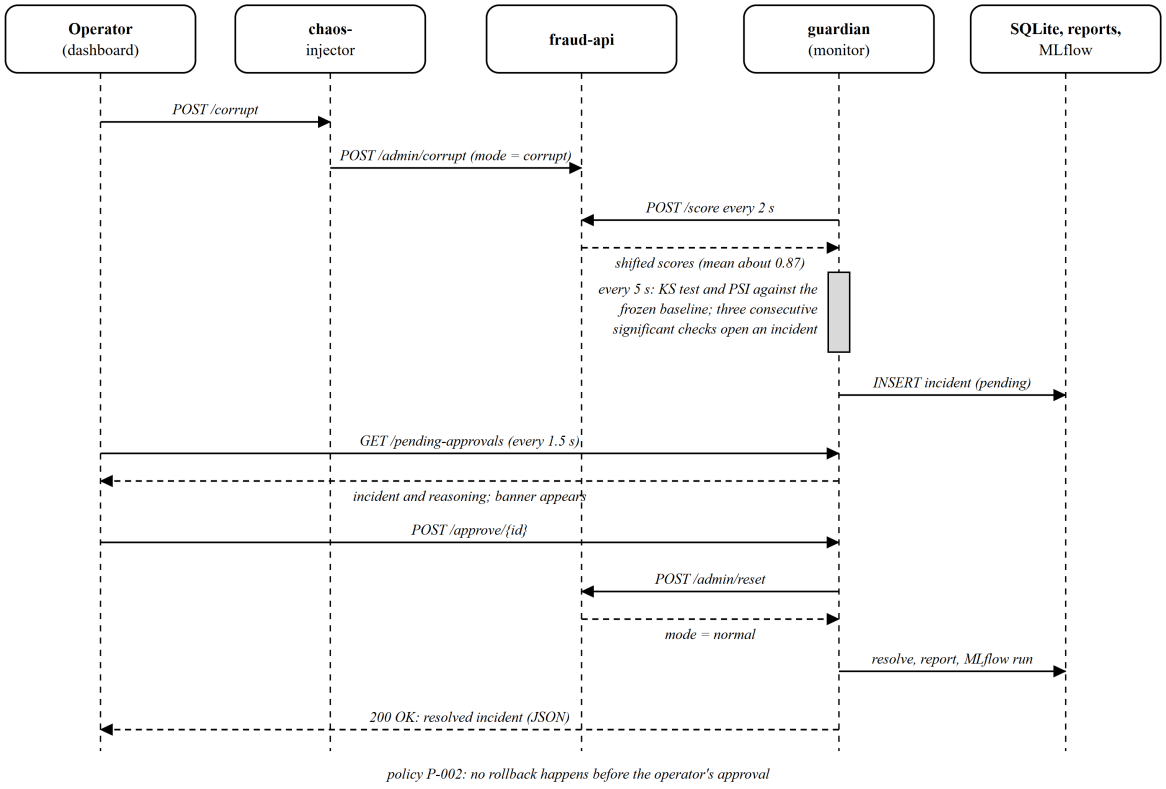


Figure 3.4: Request and response flow of a silent-failure incident

Table 3.2 states the contract of the routes that change state or enforce a rule.

Table 3.2: Key API endpoints and their observed behaviour

Endpoint	Input (JSON)	Responses
POST /score (fraud-api)	amount, merchant_category, hour, country_risk; or empty body for a random sample	200 with score, label and model_version; 422 when a field is invalid (for example hour 25).
POST /replay (fraud-api)	none; query limit	200 with the scored rows, the confusion matrix and the metrics; 404 when the dataset is missing.
POST /approve/{id} (guardian)	none	200 with the resolved incident; 400 when the incident is unknown or already resolved; 502 when fraud-api cannot be reset, in which case the incident stays pending.
POST /config (guardian)	psi_alert, poll_interval_s, drift_check_interval_s, consecutive_alerts_for_incident	200 with the applied values; values outside safe ranges are clamped and unknown keys are ignored.
GET /reports/{name} (guardian)	report file name in the path	200 with the markdown; 404 for any name that does not match incident-NNNN-DATE-TIME.md or does not exist.
POST /crash, /slow, /corrupt, /reset (chaos)	none; query ms for slow	200 with the scenario; 502 when fraud-api cannot be reached.

3.3.4 Data design

The only database is SQLite, a single-file relational store that is part of the Python standard library [15]. One table, incidents, holds one row per incident. The fields are listed in Table 3.3. Evidence such as the PSI and the KS statistic is stored as JSON text inside the row, which keeps the schema stable while the evidence evolves.

Table 3.3: Fields of the incidents table

Field	Type	Notes
id	INTEGER primary key	Assigned by SQLite; shown on the dashboard as #0001 and so on.
type	TEXT	crash or silent_failure.
status	TEXT	in_progress, pending_approval or resolved.
detected_at, resolved_at	TEXT (ISO 8601, UTC)	resolved_at is empty until the incident closes.
action	TEXT	auto_restart_observed or rollback_approved.
reasoning	TEXT	Plain-text explanation including the KS p-value, the PSI and the policy.
evidence_json	TEXT (JSON)	Drift statistics, sample counts and the live threshold.
blast_radius	INTEGER	Transactions scored while the incident was open.
mttr_seconds, report_path	REAL, TEXT	Time to recovery and the path of the markdown report.

3.3.5 Crash detection and recovery

The guardian checks /health every poll interval. One failure could be a dropped packet, so a crash is declared only after two consecutive misses, which at the default interval is about four seconds. The incident is opened in the in-progress state and the handler waits, without taking any action of its own, for the service to answer again; the restart itself is done by the container runtime (restart policy on-failure in the compose file) or by the operator when running locally. The elapsed time is stored as the time to recovery. This is why the report says that Guardian observes and times the recovery under P-001 and does not perform it. A limitation follows: a container that restarts within one poll interval would not produce two misses, and the Docker behaviour was not exercised in this environment (Table 4.4).

3.3.6 Silent-failure detection: the drift rule

The scorer's output is a number in [0, 1], so drift is detected by comparing the distribution of recent scores with a frozen baseline. Two statistics are used. The two-sample Kolmogorov-Smirnov test gives a p-value for the hypothesis that the two samples come from the same distribution [3], [16]. The Population Stability Index sums, over ten bins whose edges are the baseline deciles, the quantity (a - e) ln(a / e), where a and e are the actual and expected fractions of samples in the bin [11]. PSI is a size measure that risk teams already know, with 0.10 and 0.20 as the usual warning and alert levels.

The first version used PSI alone: an incident opened when PSI reached 0.20 on two consecutive checks. Testing showed that this was wrong. PSI computed from a window of 60 samples is biased upward, and for healthy traffic its mean is about 0.32, above the alert level, so the rule fired on a healthy system. Three incidents were in fact opened on healthy traffic with KS p-values between 0.27 and 0.75. The current rule uses the KS test as a gate and PSI as a size filter. A check is significant when the KS p-value is below 1e-4 and PSI is at least the alert threshold; it is minor when p is below 1e-2 and PSI is at least the warning threshold; otherwise it is stable. An incident opens after three consecutive significant checks. Figure 3.5 shows the control flow.

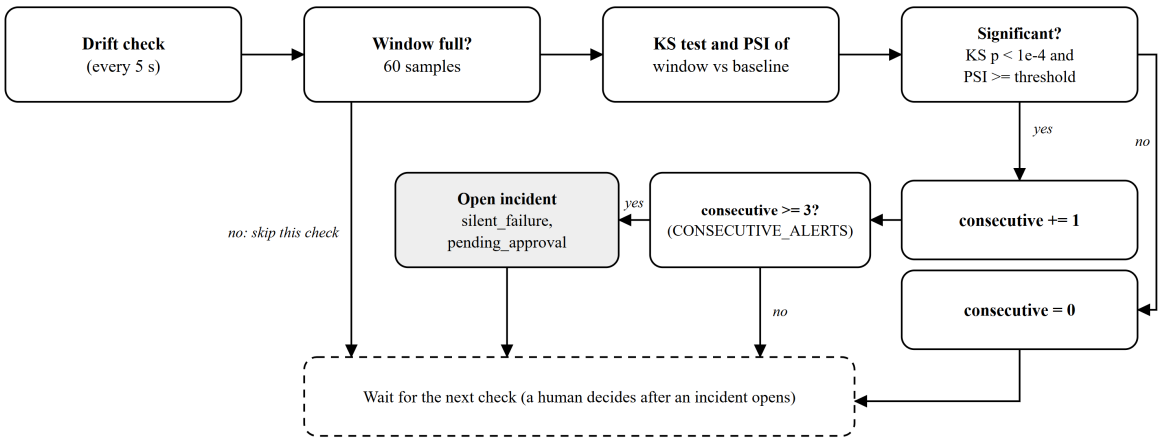


Figure 3.5: Control flow of the drift check

The confidence gauge shown on the dashboard is derived from the same test: it falls linearly with log10 of the KS p-value, from 100 when p is 1 to 0 when p is 1e-4 or lower. Because it comes from the KS p-value and not from PSI, a healthy system shows a high gauge and a corrupted one shows zero.

3.3.7 Governance and approval

Policy P-002 is implemented as a state, not a comment. A silent-failure incident is created with the status pending_approval, and the only code path that calls the reset of fraud-api is the approve route. That route refuses unknown and already resolved incidents with HTTP 400, and it treats a failure of the reset call as an error: it returns HTTP 502 and leaves the incident pending, so the audit trail never claims a rollback that did not happen. The end-to-end scenario asserts the absence of an automatic rollback by waiting six seconds after the incident opens and checking that fraud-api is still in corrupt mode.

3.3.8 Post-mortem generation and the MLflow ledger

When an incident resolves, the guardian renders a markdown post-mortem containing the classification, timeline, detector, action, blast radius, time to recovery, governing policy, status and reasoning, and logs an MLflow run named after the incident with tags for type, status and action, metrics for time to recovery and blast radius, and the report as an artifact [4]. MLflow is normally used for model experiments; here it is deliberately reused as an incident ledger that can be filtered by tag. A failure of the MLflow server is logged and does not block the incident from resolving.

3.3.9 Observability

The fraud-api exports a request counter by outcome, a latency histogram and a gauge for the operating mode. The guardian exports gauges for the confidence score, the PSI and whether fraud-api is up. Prometheus scrapes both every five seconds, and Grafana provisions a data source and a dashboard from files, so the observability stack needs no manual set-up [5], [6].

3.3.10 Runtime configuration

The dashboard's settings panel calls POST /config. The guardian applies the change to its live configuration, clamps each value to a safe range, keeps the warning threshold at or below the alert threshold, and ignores unknown keys. An earlier version stored the threshold but the drift function ignored it; the function now takes the threshold as an argument and a test proves that changing it changes the verdict.

3.3.11 Security considerations

GuardianOps is a laboratory system and is not hardened for a hostile network. Several measures are present: the report endpoint accepts only names that match a strict pattern and then checks that the file exists, the configuration endpoint clamps its inputs, and the service accepts only validated transaction fields. Several measures are absent and are recorded as future work: there is no authentication or authorisation on the approve and admin routes, cross-origin requests are allowed from any origin, and nothing rate-limits the endpoints. This report does not claim a security review.

3.3.12 Build, launch and continuous integration

The Windows batch launcher creates a virtual environment, installs the pinned requirements, refuses to start when one of the ports is already in use, starts the four processes in titled windows, waits until all four answer and then opens the dashboard. A matching stop script closes the windows by title. For hosts with Docker, docker-compose.yml defines seven services. The GitHub Actions workflow runs on pushes, pull requests and manual dispatch.

4. WORK DONE

This chapter documents the work itself. It begins with the development environment, then describes each implementation module, then reports the results of verification and finally records my individual contribution. Statements about what was implemented are tied to commits in the public repository so that they can be checked.

4.1 Development Environment

Table 4.1 lists the technology stack with the role of each item. Versions are those pinned in the requirements files or in docker-compose.yml.

Table 4.1: Technology stack and responsibilities

Technology	Version (pinned)	Role in the project
Python	3.11	Language of all three services and the tooling.
FastAPI, Uvicorn	0.115.0, 0.30.6	Web framework and server for the three services [2].
Pydantic	2.9.2	Request validation (for example hour between 0 and 23).
HTTPX	0.27.2	Asynchronous HTTP client used by guardian and chaos-injector.
NumPy, SciPy	1.26.4, 1.13.1	Array handling and scipy.stats.ks_2samp for the KS test [3].
prometheus-client	0.20.0	Counters, histograms and gauges on /metrics.
MLflow (skinny client; server image)	2.14.1	Incident ledger [4].
SQLite	bundled with Python	Incident store [15].
Prometheus, Grafana	2.53.0, 11.1.0	Metrics collection and dashboards [5], [6].
nginx	1.27 (alpine)	Serves the dashboard in the compose stack.
Docker Compose	v2 syntax	Seven-service deployment [7].
pytest	8.x	Unit and API tests [9].
Playwright (Edge)	1.x	Automated, scripted screenshots of the live dashboard at 2x scale [14].
GitHub Actions	-	Continuous integration [8].
Git, GitHub	-	Version control and public hosting [13].

Working procedure. On Windows the project is started with start-guardianops.bat, which prepares a virtual environment and opens the four processes; docker compose up --build starts the full stack on any machine with Docker. The services recompile nothing and reload on restart, which made the running system the main debugging aid. The tests are run with pytest inside each service folder, and audit.sh and scripts/e2e_scenario.py are run against the live processes. The repository is in the public GitHub account pradhanashwarya2122 and the commands are listed in the README.

4.2 Implementation Modules

4.2.1 Module 1: The monitored service (fraud-api)

The fraud-api exposes POST /score, which accepts a transaction or draws a random one, and returns a score, a label and the model version rules-1.0. The score is a weighted sum of an amount signal, an hour-of-day signal, a merchant-category signal and a small noise term, plus a country-risk term; scores above 0.7 are labelled fraud and above 0.4 review. In corrupt mode the same route returns random draws from a Beta(9, 1.4) distribution, whose mean of about 0.87 makes almost every transaction look fraudulent. The route /replay streams the bundled file data/replay/transactions.csv, 500 seeded synthetic transactions of which 20 are labelled fraud in four patterns (large overnight purchases from high-risk countries, ATM bursts abroad, high-value online purchases at night and small card-testing payments), through the same scoring function and returns a confusion matrix. Figure 4.1 shows the dashboard in its healthy state with the live trace of this service.



Figure 4.1: Dashboard in the healthy state (local run, commit 54b90c8)

4.2.2 Module 2: Monitor and drift detection (guardian)

The guardian captures a baseline of 200 scores when it starts, then loops every two seconds: it checks health, draws a score into a rolling window of 60 and, every five seconds, runs the drift check when the window is full. A half-full window is skipped on purpose, because a small window inflates PSI. Section 3.3.6 gives the rule. The main finding of the project was made here. Testing the system interactively showed incidents #3 to #5 opening on a healthy system, and an offline simulation of 200 healthy hours confirmed that the first rule raised at least one false incident in every hour. The replacement rule reduced this to 2 hours out of 200 (1.0%). Figure 4.2 shows the crash case.



Figure 4.2: A crash: no-signal region on the trace, incident and decision log (local run)

4.2.3 Module 3: Governance and the incident lifecycle

When the drift rule fires, the guardian inserts an incident with the status pending_approval and the dashboard raises a banner, shown in Figure 4.3, and an approval card. The card opens with a plain-English summary and offers collapsible sections: what drift means, the numbers behind the decision, why Guardian asks before it rolls back, and the raw reasoning. Figure 4.4 shows the card with the first two sections open. The approval action resets fraud-api, sets the incident to resolved with the action rollback_approved, stores the time to recovery, renders the post-mortem (Figure 4.5) and logs the MLflow run.

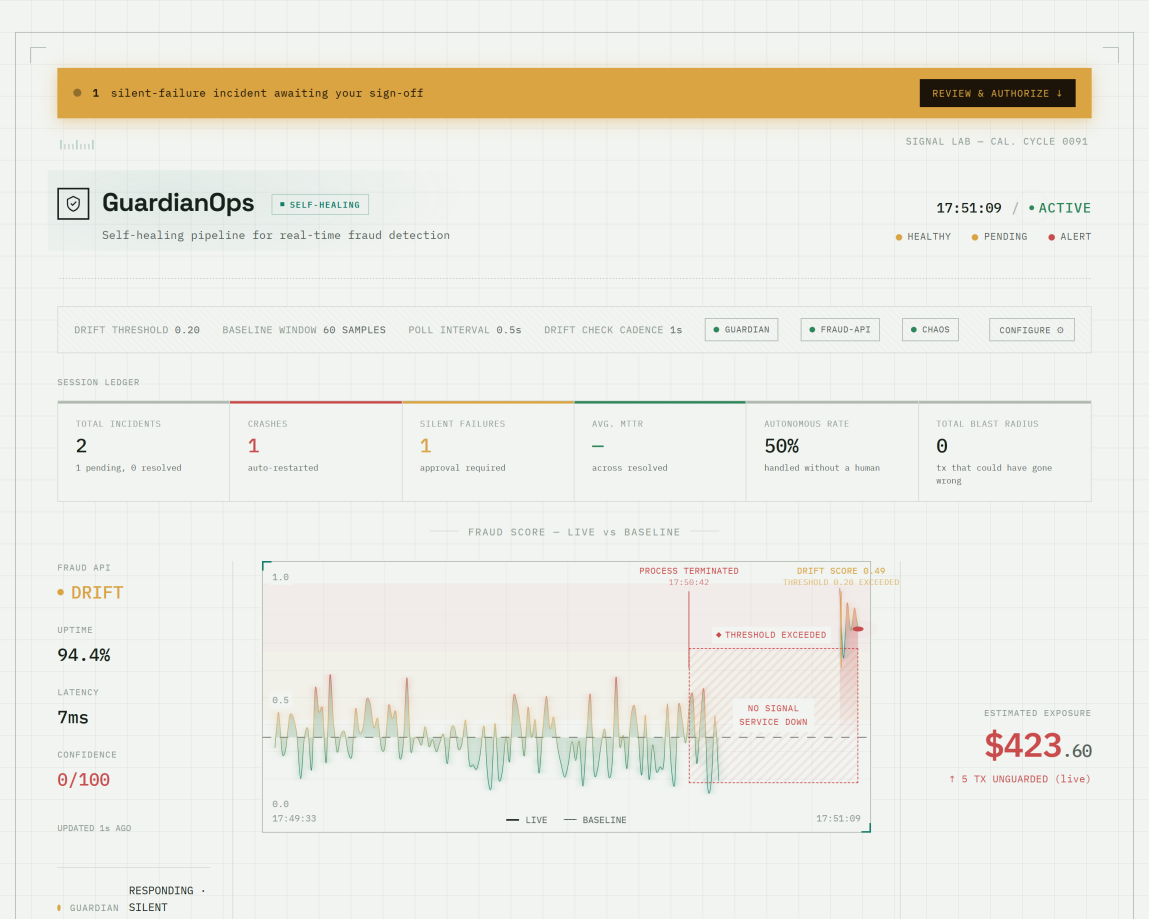


Figure 4.3: Silent-failure incident awaiting approval (banner, ledger and trace; local run)

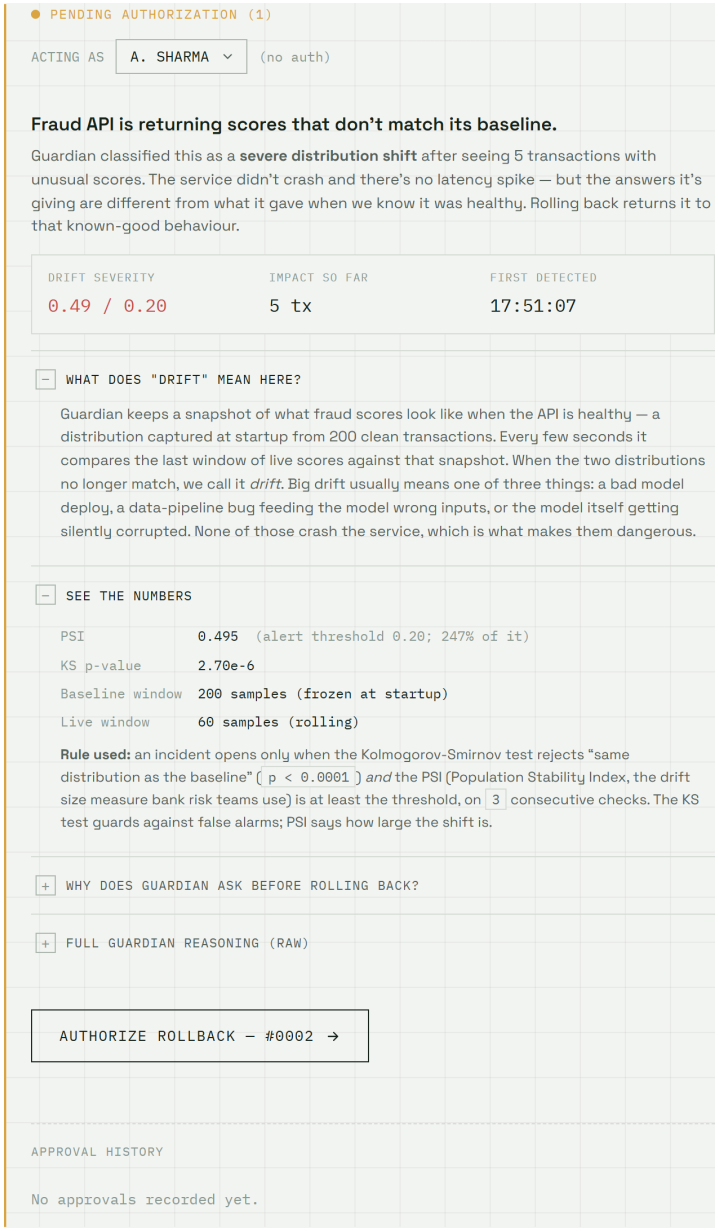


Figure 4.4: Approval card with the first two sections expanded (local run)

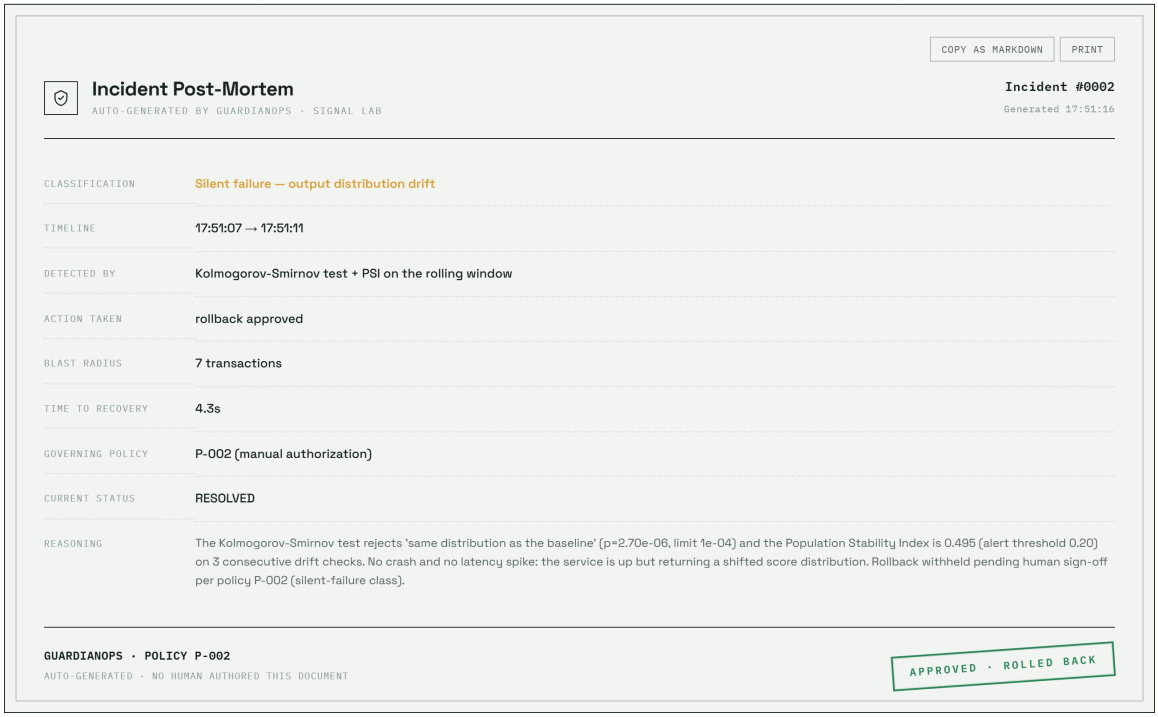


Figure 4.5: Auto-generated incident post-mortem after approval (local run)

4.2.4 Module 4: Evaluation and test tooling

Four artefacts verify the system. The unit tests (38 in all) check the scoring bands, input validation, the effect of corruption and reset, the confusion matrix on a four-row file, the drift rule, the clamping of the configuration and the failure handling of the approval route. The audit script audit.sh makes 35 checks of the live endpoints. The scenario script scripts/e2e_scenario.py drives the complete silent-failure path. The tool services/guardian/tools/evaluate_drift.py simulates healthy and corrupted traffic and measures the false-alarm rate and detection delay of the old and new rule. Figure 4.6 shows the workflow used when any of them failed.

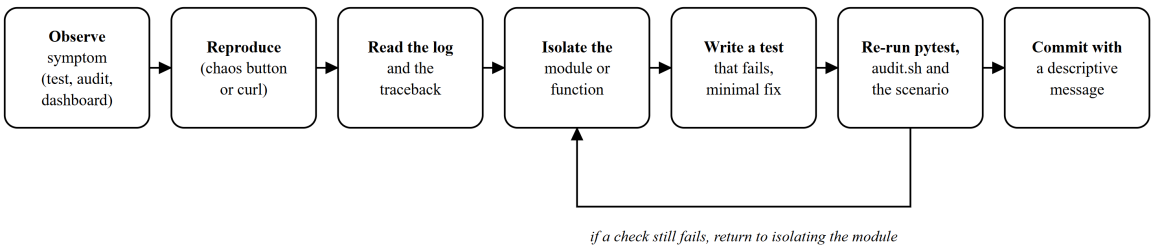


Figure 4.6: Development and debugging workflow used during the project

Table 4.2 lists the defects that are evidenced by commits or by tests. They are reported here rather than hidden because the purpose of triage is to find problems.

Table 4.2: Defect log: symptom, cause and action

Ref.	Symptom	Cause and action	Evidence
D1	Incidents opened on a healthy system.	PSI of a 60-sample window is biased upward (mean about 0.32). Replaced the rule by a KS gate, a PSI size filter and three consecutive checks.	Commit b2fd9e6; test_drift_rule.py
D2	The batch launcher failed or misbehaved on Windows.	A trailing space in an environment variable (set X=a && cmd), LF-only line endings and non-ASCII characters. Rewritten with set "VAR=value", CRLF, ASCII; .gitattributes added.	Commit b2fd9e6
D3	POST /approve returned HTTP 500 when fraud-api was down.	The reset call was not guarded. Now HTTP 502 and the incident stays pending.	test_api.py (approve with fraud-api down)
D4	A post-mortem printed the word None for open fields.	Unresolved timestamps and actions were formatted blindly. Made None-safe.	test_api.py (report has no None)
D5	The report endpoint accepted any file name.	Names are validated against a pattern before the filesystem is touched.	audit.sh (traversal returns 404)
D6	Replay reported 1001 rows instead of 500.	The CSV had CR CR LF endings created by a shell redirect on Windows. Generator now writes the file directly with LF; row count is robust.	Commit b2fd9e6
D7	The CI workflow never ran.	It listened to main while the default branch is master. Now main and master, pull requests and manual runs.	.github/workflows/ci.yml
D8	The configured PSI threshold had no effect.	compute_drift ignored the run-time value. The threshold is now a parameter.	test_api.py (threshold decides verdict)
D9	Interface faults seen in screenshots.	An unstyled NO SIGNAL label was always visible, the report buttons overlapped the incident number and the replay dialog called synthetic data real transactions. Fixed in the dashboard.	Commit 54b90c8

4.2.5 Module 5: Failure injection (chaos-injector)

The chaos-injector offers four routes. /crash asks fraud-api to terminate its process, /slow adds a fixed delay to every score, /corrupt switches the scoring function to the shifted distribution, and /reset restores normal mode. The injector does not change fraud-api's code. It calls the admin routes, so the same faults can be provoked by a script, by curl or by the dashboard. The audit confirms that each injected mode really appears in the fraud-api health response.

4.2.6 Module 6: Dashboard

The dashboard polls the guardian every 1.5 seconds. Its twelve blocks (Figure 3.3) include the session ledger, whose six tiles are computed from the incident table, the score trace with callouts for incidents, the controls for the three faults, the forced drift check and the replay, and the decision log that narrates what the guardian does. The replay control opens the modal shown in Figure 4.8, in which the

confusion matrix and the metrics come from the live endpoint. The settings panel changes the guardian's thresholds through POST /config.

4.2.7 Module 7: Observability stack

Prometheus is configured to scrape fraud-api and guardian, and Grafana is provisioned with the data source and a dashboard file. The configuration files were validated with docker compose config. The stack was not started, because the container engine did not respond in the verification environment; the dashboards were therefore not inspected and no screenshot of them is included.

4.2.8 Module 8: Build, launch and continuous integration

Figure 4.7 shows the delivery flow. A push triggers the workflow, which runs pytest for both services, boots the services and runs the audit and the end-to-end scenario, and builds the three images. The same script, scripts/ci_smoke.sh, can be run locally. For a local demonstration the batch launcher replaces Docker by four processes.

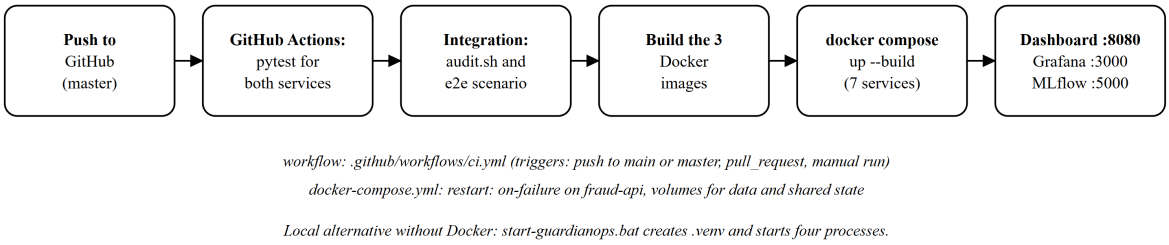


Figure 4.7: Build and deployment flow defined by the workflow and the compose file

4.3 Results and Discussion

Verification was performed on 25 October 2026 (audit and end-to-end run on commit b2fd9e6, which differs from the final commit only in dashboard fixes, one added test and documentation) on a Windows 11 machine, with the services run as local processes from the pinned requirements. The numbers below are copied from the command output saved in report/results/verification_run.txt and from the evaluation file report/results/drift_evaluation.json.

Table 4.3: Verification performed and observed results

Check	Req.	Observed result
pytest in services/fraud-api	FR1, FR2, FR8	16 passed.
pytest in services/guardian	FR3 to FR7	22 passed.
audit.sh against the live services	NFR4, FR7	35 checks passed, 0 failed, 1 skipped (the single-report read, skipped because no report existed yet).
End-to-end scenario (e2e_scenario.py)	FR3, FR5, FR6	Passed. No incident in 40 s of healthy traffic; corruption opened a pending incident (PSI 0.52, KS p 5.9e-07); no automatic rollback after six seconds; approval resolved it in 6.5 s; the report contained no None; a second approval returned HTTP 400.
Replay of 500 labelled transactions	FR8	TP 5, FP 0, TN 480, FN 15; precision 1.0, recall 0.25, F1 0.40, accuracy 0.97.
Healthy-traffic simulation, old rule	NFR1	False incident in 200 of 200 simulated hours (100%).
Healthy-traffic simulation, current rule	NFR1	False incident in 2 of 200 simulated hours (1.0%).
Detection of Beta(9, 1.4) corruption, current rule	FR3	Detected in 100% of runs; median 54 s.
Detection of a +0.10 score shift	FR3	Detected in 99% of runs; median 126 s.
Detection of a +0.05 score shift	FR3	Detected in 21% of runs.
Confidence gauge, healthy vs corrupt	FR9	Healthy median 94.7 (5th percentile 77.8); corrupt 0.
docker compose config -q	NFR6	Valid. The images were not built and the stack was not started: the container engine did not respond.
GitHub Actions workflow status	NFR7	The status badge of the ci workflow on master read passing when checked on 25 October 2026; the individual run page was not inspected.

The pattern of results is informative. The detector is now conservative: it almost never fires on healthy traffic, it catches a severe corruption within a minute, and it is slow or blind to small shifts, a trade-off that follows directly from requiring both statistical significance and three consecutive checks. The scorer is a different matter. With recall of 0.25, the rule-based scorer catches only 5 of the 20 fraud rows in the replay file, and that weakness is a property of the hand-written rules and not of the monitoring layer. Finally, the earlier overclaim that the audit covered every endpoint was corrected: the first audit probed 21 of the 30 routes, and it now makes 35 checks that include the write endpoints and the negative cases.

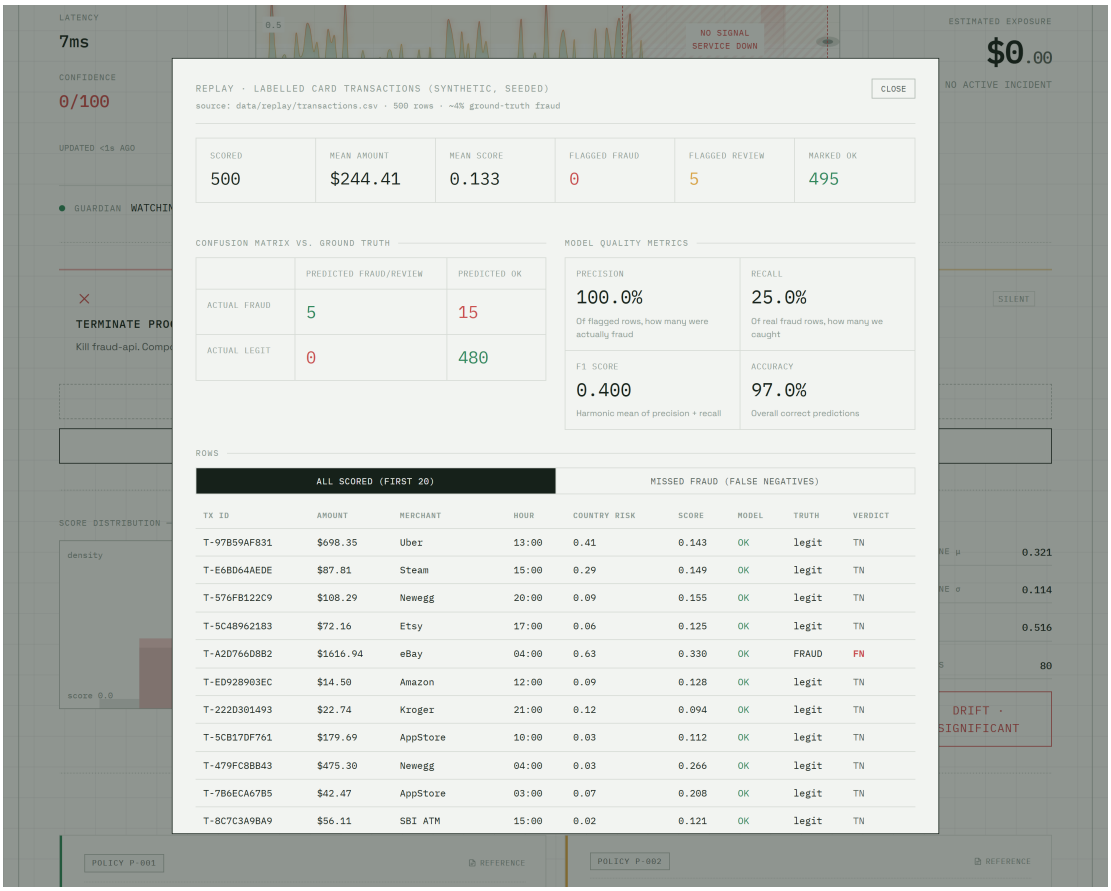


Figure 4.8: Replay of 500 labelled transactions: confusion matrix and metrics (local run)

Two behaviours deserve comment as design observations. First, the approval label on the dashboard (an operator selector) is not authentication, so the approval policy is enforced by workflow and audit only. Second, because the baseline is captured at start-up, the guardian must start while fraud-api is healthy; a baseline captured during a fault would hide it, which is why fraud-api resets to normal mode at start-up.

Table 4.4: Limitations identified and suggested improvements

Limitation	**Consequence**	**Suggested improvement**
The scorer is a rule set with 25% recall on the replay file	It misses most injected fraud patterns.	Train and register a real model; monitor it with the same layer.
The approval action has no authentication	Anyone who can reach the port can approve.	Add authentication and record the authenticated user in the incident.
Small shifts are detected slowly or not at all	A +0.05 shift is found in 21% of runs.	Add a CUSUM-type test or longer windows; make the trade-off configurable.
Crash detection needs two missed polls	A container that restarts within one interval may be missed.	Expose a boot identifier in /health and detect its change.
The Docker stack was not run in the verification environment	Compose behaviour, dashboards and the MLflow artifact upload under compose are untested.	Run the stack, check restart on failure and the artifact root, and record screenshots.
Data are synthetic	No claim about real fraud rates or real drift.	Evaluate on a public labelled dataset with the same replay route.
Mode is shared through a JSON file	Services must share a volume; one writer at a time.	Use a small shared store or an admin API with its own state.

4.4 Individual Contribution of Project Members

This was an individual project. All commits in the repository were made under one author account, and the work in each module was done by me. The tools used while working included an AI coding assistant, and the final design decisions, the verification and the reporting of results were mine to make and check. Table 4.5 maps the modules to the commits that contain them.

Table 4.5: Mapping of my contribution to modules and evidence

Module	**My contribution**	**Evidence**
Services and dashboard	Initial services, dashboard, scope visualisation, runtime configuration, replay dataset and approval card.	Commits 221b3e7 to 9cb98cc
Documentation	README, audit script, roadmap and presentation notes.	Commits 3994df8, 5525ab2, bd5e391
Report package and launcher	First report, screenshots and one-click launcher.	Commit 2bd1df2
Detector, API hardening, tests, CI, launcher, dataset and report	New drift rule, hardened approve and report routes, 38 tests, audit and end-to-end scenario, CI integration job, launcher rewrite, evaluation tool and this report.	Commit b2fd9e6
Dashboard fixes, wording, regression test	Removed the stray NO SIGNAL label, moved the chart legend, fixed the report toolbar, corrected the synthetic-data wording, added the replay row-count test and the high-resolution screenshots.	Commit 54b90c8
Triage	Reproduced, classified and fixed the defects in Table 4.2.	Defect log

I also learned from this work. The most important lesson was that a monitoring system can be wrong in a way that looks like correctness: the first detector produced convincing alerts, and only a test that checked its behaviour on healthy data revealed that it could not tell a healthy system from a broken one. In future I would write the statistical expectation as a test before writing the detector.

5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

The project delivered a working platform that distinguishes a crash from a silent failure in a fraud-scoring service, responds to each under a written policy, and leaves an auditable record. Thirty-eight automated tests, a 35-check endpoint audit and an end-to-end scenario passed on the final code, and every figure in this report comes from a run or a file that is in the repository.

The project also taught me the discipline of verification. Checking the system showed that the first detector raised a false alarm in every simulated healthy hour, that an approval could fail with a server error, that the launcher and the continuous-integration workflow did not do what they claimed, and that several interface claims were not true. I have reported these findings plainly because they are the kind of finding that engineering practice exists to produce, and because each one has a remedy in the repository.

Table 5.1 summarises the technical and professional skills that the project developed, each tied to the work that exercised it.

Table 5.1: Skills and learning outcomes

Skill	**Where it was developed**
Designing and implementing REST services with FastAPI and validated input	Modules 1, 3 and 5.
Statistical monitoring of model outputs (KS test, PSI) and its failure modes	Module 2, Section 3.3.6.
Governance of automated actions and audit trails	Module 3, Section 3.3.7 and 3.3.8.
Observability with Prometheus, Grafana and MLflow	Module 7, Section 3.3.9.
Test design: unit, API, audit and end-to-end tests	Module 4; 38 tests, 35 audit checks.
Debugging across processes and operating systems	Defects D2, D6 and D7 in Table 4.2.
Continuous integration and reproducible builds	Module 8.
Honest reporting of limitations	Section 4.3 and Table 4.4.

5.2 Future Scope of Work

The following improvements follow from the limitations in Table 4.4. They are proposals and have not been implemented.

Table 5.2: Proposed future work

No.	Proposal	Expected benefit
1	Run the Docker stack, verify restart on failure, Prometheus targets, the Grafana dashboard and the MLflow artifact upload; add their screenshots.	The deployment claims are verified.
2	Add a boot identifier to /health so that fast restarts are detected as crashes.	No crash is missed.
3	Add authentication to the approve and admin routes and store the approver.	The approval is accountable.
4	Replace the rule-based scorer by a trained model evaluated on a public labelled dataset.	Realistic quality figures.
5	Add a sequential test for small shifts and expose the detection trade-off in the dashboard.	Faster detection of small drift.
6	Replace the shared state file by a small shared store.	Multi-instance deployment.

5.3 Proposed Workplan of the Project

Table 5.3 proposes a workplan for the future work, expressed in weeks from the start of the next phase. The durations are estimates and not commitments.

Table 5.3: Proposed workplan for the next phase

Week	Activity	Deliverable
1	Run the compose stack end to end; fix compose or MLflow settings if needed.	Verified deployment and screenshots.
2	Boot identifier in /health; crash-detection tests.	Fast restarts detected.
3	Authentication and approver identity on the approve route.	Accountable approvals.
4	Public dataset; train and register a model; replay evaluation.	Realistic metrics.
5	Sequential drift test; evaluation of detection delay.	Updated evaluation file.
6	Documentation review and release notes.	Tagged release.

REFERENCES

[1] S. Ramirez, "FastAPI Documentation," fastapi.tiangolo.com. Last accessed: 25 October 2026.

[2] SciPy community, "scipy.stats.ks_2samp," docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html. Last accessed: 25 October 2026.

[3] MLflow project, "MLflow Documentation," mlflow.org/docs/latest. Last accessed: 25 October 2026.

[4] Prometheus Authors, "Prometheus Documentation," prometheus.io/docs. Last accessed: 25 October 2026.

[5] Grafana Labs, "Grafana Documentation," grafana.com/docs. Last accessed: 25 October 2026.

[6] Docker Inc., "Docker Compose Documentation," docs.docker.com/compose. Last accessed: 25 October 2026.

[7] GitHub, "GitHub Actions Documentation," docs.github.com/actions. Last accessed: 25 October 2026.

[8] pytest-dev, "pytest Documentation," docs.pytest.org. Last accessed: 25 October 2026.

[9] D. Sculley et al., "Hidden Technical Debt in Machine Learning Systems," in Advances in Neural Information Processing Systems 28 (NeurIPS 2015), papers.nips.cc.

[10] N. Siddiqi, "Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring," John Wiley & Sons, 2006 (the Population Stability Index in scorecard monitoring).

[11] R. Fielding, M. Nottingham and J. Reschke (eds.), "HTTP Semantics," RFC 9110, IETF, June 2022, www.rfc-editor.org/info/rfc9110.

[12] Git, "Git Documentation," git-scm.com/doc. Last accessed: 25 October 2026.

[13] Microsoft, "Playwright for Python Documentation," playwright.dev/python. Last accessed: 25 October 2026.

[14] SQLite Consortium, "SQLite Documentation," sqlite.org/docs.html. Last accessed: 25 October 2026.

[15] F. J. Massey, "The Kolmogorov-Smirnov Test for Goodness of Fit," Journal of the American Statistical Association, vol. 46, no. 253, pp. 68-78, 1951.